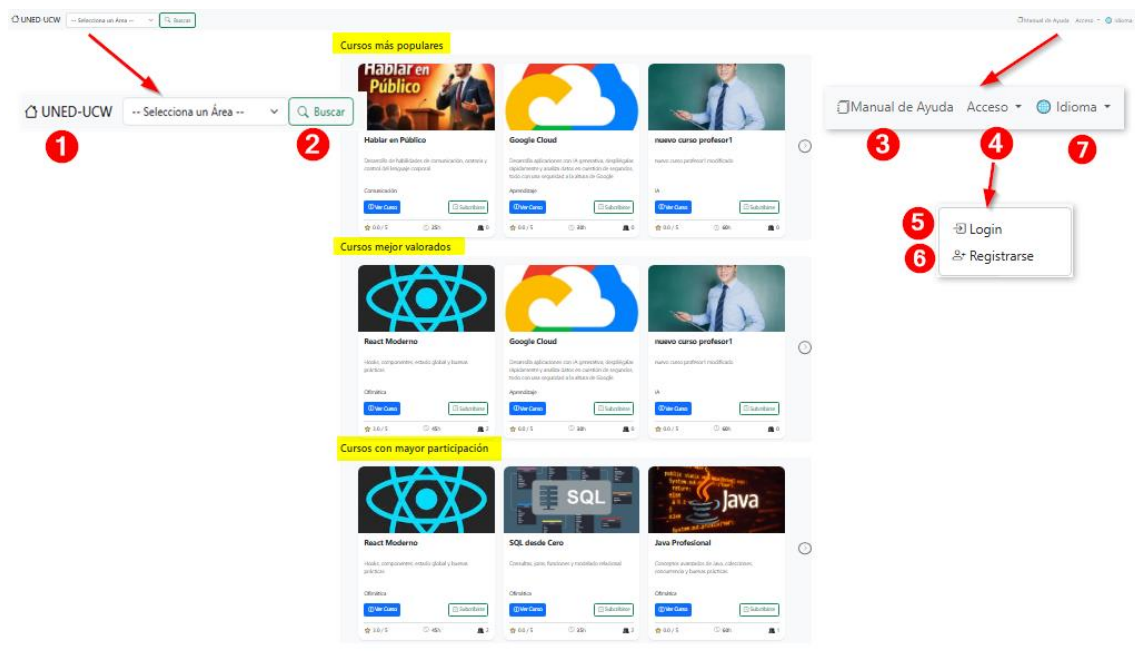


MANUAL DE USUARIO

Pantalla inicial

Al acceder a la aplicación se visualizarán la home pública



1.- Link a la Home. Desde cualquier parte de la aplicación se podrá volver a la “home” clickando en este apartado.

2.- Buscador. El buscador estará disponible en todo momento (para usuarios anónimos y estudiantes), mostrándonos las áreas temáticas incluidas en el sistema.

3.- Manual de ayuda. Nos mostrará este documento en el que se detallará la operativa del sistema.

4.- Acceso. (usuario no identificado). Nos ofrecerá las opciones de:

5.- Login. Identificarse en la plataforma con los datos de nuestro usuario.

6.- Registrarse. Darse de alta en el sistema si no tenemos usuario.

7.- Selector de idioma. Al desplegar la opción se podrá seleccionar uno de los dos idiomas disponibles (Español e Inglés).

Zona central con la visualización de los cursos existentes en la plataforma, divididos en 3 “carruseles”:

- Cursos más populares. Ordenados por el número de visitas.
- Cursos mejor valorados. Ordenados por la media de valoración obtenida.
- Cursos con mayor participación. Los cursos con mayor número de estudiantes inscritos.

Login

Es la pantalla obligatoria para operar con la aplicación.

Se accede desde la opción del menú superior “Acceso” -> “Login”

The image shows a login form titled "Acceso". It contains two input fields: "Usuario" with the email "manuel.perez@correo.es" and "Contraseña" with masked characters. Below the fields is a blue button labeled "Acceder" with a key icon. At the bottom, there is a link that says "¿No tienes cuenta? Regístrate".

Se introducirá los datos de “Usuario” (e-mail) y “Contraseña” (Password).

Al pulsar sobre “Acceder”, si los datos son incorrectos, recibiremos el mensaje:

Acceso

Usuario o contraseña Incorrectos

Otra posibilidad es que el usuario esté “bloqueado”, en este caso el mensaje recibido será:

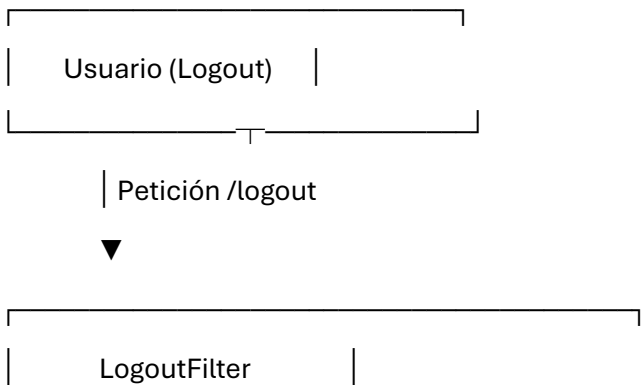
Acceso

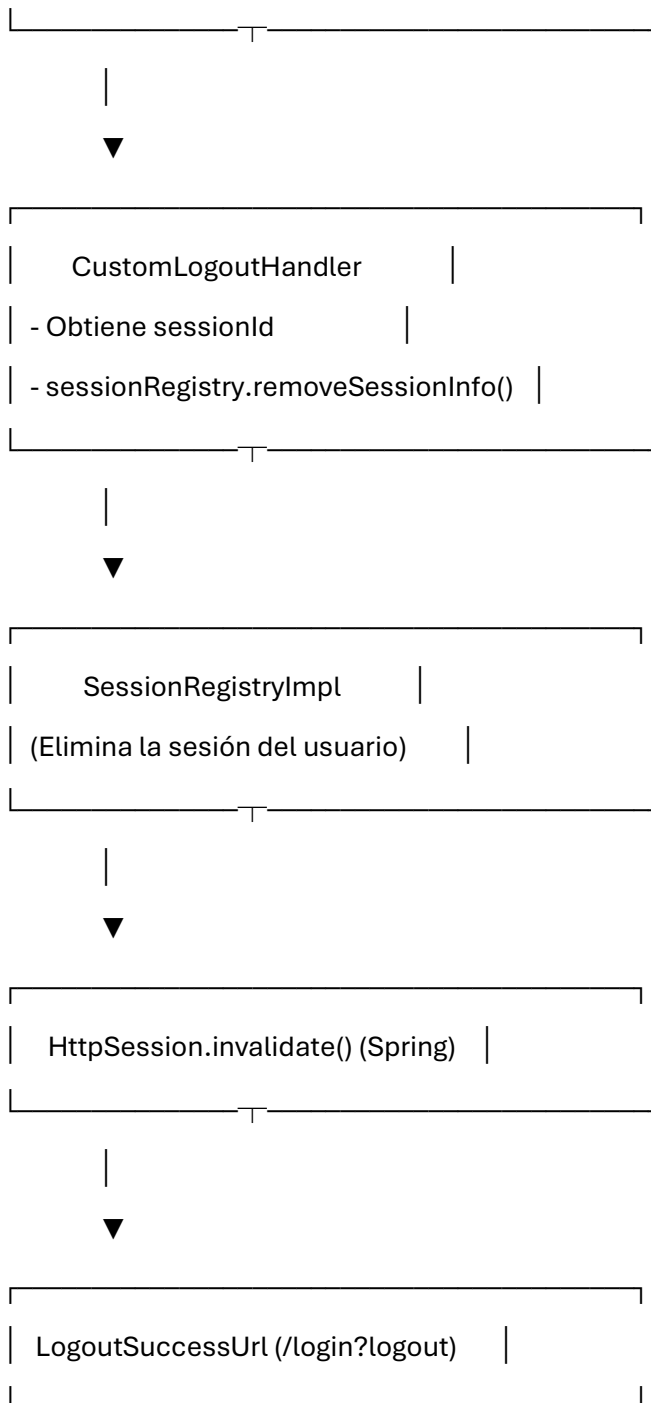
Su cuenta está desactivada. Por favor, contacte con soporte.

Proceso de registro de usuarios en sesión:



Proceso de logout y eliminación del registro de usuarios en sesión:





Validación de datos y excepciones

Se implementa la validación de datos en las vis

Errores en vistas

Errores específicos

Errores globales

A nivel de formularios y vistas en la aplicación se añade un fragmento:

Errorgeneral.html

```
<div th:fragment="errorgeneral">
    <div th:if="${errorGlobal}" class="alert alert-
danger">
        <span th:text="${errorGlobal}"></span>
    </div>
</div>
```

Que será incluido en todas las vistas que lo precisemos.

Desde el controlador añadiremos este atributo cuando se considere necesario:

```
model.addAttribute("errorGlobal", MENSAJE_DE_ERROR_A_MOSTRAR);
```

PATRONES UTILIZADOS

MVC

BUILDER

PRG (Post-Redirect-Get)

- DTO con @Valid

En las clases editables en los formularios DTO, se generan validaciones a nivel de campo/atributo mediante anotaciones “jakarta.validation.constraints”, del paquete “jakarta.validation.api” que incorporan las más utilizadas.

@NotBlank Para campos String, controla si es nulo o sólo espacios en blanco.

@NotNull Para campos numéricos y de fecha.

@Size Tamaño, mínimo ó máximo.

@Email Comprueba si el formato es un correo electrónico.

@Positive Número positivo

@PositiveOrZero Positivo ó cero.

@FutureOrPresent Para fechas, que el valor sea mayor o igual al día de hoy.

```
public class ClaseDTO {

    @NotBlank(message = "{validations.mensaje1}")
    private String atributo1;

    @NotNull(message = "{validations.mensaje2}")
    private Integer atributo2;
```

```

    // Resto de código
}

```

Esta implementación nos facilita las comprobaciones y los valores condicionales en nuestro código, permitiéndonos definir nuestro mensaje mediante claves de mensajes que incluiremos en los ficheros de “internacionalización (i18n)” definidos. Sólo algunos valores “no anotados” deberán comprobarse en nuestro código (normalmente en el “Controller”), de forma manual.

La validación se realiza mediante dos mecanismos:

Jakarta.validation.valid

Se anota el atributo del DTO que llega desde el formulario como @Valid

```

@PostMapping("/accion")
public String guardarDatos(@AuthenticationPrincipal UserDetails userDetails,
    @Valid @ModelAttribute ClaseDTO dto,
    BindingResult result,
    Model model)

```

org.springframework.validation.beanvalidation.SpringValidatorAdapter.SpringValidatorAdapter

Este SpringValidator lo empleamos cuando para un mismo DTO, las validaciones sean diferentes según las necesidades que tengamos identificadas.

Tendremos que eliminar el @Valid de nuestros parámetros y diferenciar el comportamiento de las validaciones anotadas añadiendo “groups” a la anotación en los DTO’s y que corresponderán con “Interfaces” que se deberán implementar.

```

public interface OnCreate extends Default {

}

public interface OnUpdate extends Default{

}

```

En el DTO, se anotará:

```

@NotBlank(message = "{validations.message.password.mandatory}", groups =
OnCreate.class)
@Size(min = 6, message = "{validations.message.password.validformat}", groups =
OnCreate.class)
private String confirmPassword;

```

Y en nuestro método del controller se implementa como:

```

// 1) Para utilizar el mismo DTO
// Creación (con password)
// Modificación (sin password) y que no salte el @NotBlank del
validator
Class<?> grupo = (form.getId() == null) ? OnCreate.class :
OnUpdate.class;

```

```
// 2) Validar dinámicamente según el grupo
    new SpringValidatorAdapter(validator)
        .validate(form, result, grupo);
```

Con este mecanismo y este ejemplo, se validará el campo “confirmPassword”, sólo si el “id” no existe, asignando al “validator” el grupo “OnCreate”, es decir, cuando es NUEVO. En los casos de modificación, se asigna el grupo “OnUpdate” y el validador obviará los asignado a otros grupos.

Por defecto, todos los atributos están identificados con el grupo “Default” que ya incorpora Spring.

Resultado de la validación

Si alguna de las validaciones no se cumple, por cualquier de los dos mecanismos descritos anteriormente, se generan los mensajes asociados a cada validación y devolviéndose a la vista.

Para ser mostrado, se añaden al formulario unos “divs” que son interpretados por “Thymeleaf” en nuestro caso, y que sólo se mostrarán cuando existan.

Ejemplo de implementación:

```
<div class="text-danger" th:errors="*{atributo1}"></div>
```

- BindingResult

En el método POST se indica que se realice el “Binding” entre las validaciones definidas en el DTO y los valores que nos llegan de la vista.

```
@PostMapping("/accion")
public String guardarDatos(@AuthenticationPrincipal UserDetails userDetails,
    @Valid @ModelAttribute ClaseDTO dto,
    BindingResult result,
    Model model)
```

La comprobación es la siguiente:

```
if (result.hasErrors()) {
    return "vista_del_formulario";
}
```

Si hay errores en la validación, volvemos a la vista del formulario.

- Excepciones de negocio en el Service

Los datos del formulario pueden ser válidos en formato de acuerdo a lo determinado en sus atributos, pero no cumplir las reglas de negocio que tenemos en nuestro modelo, en nuestro caso en los “services”.

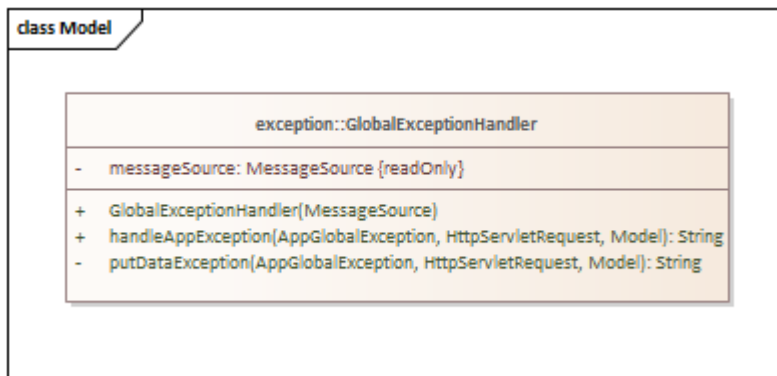
En este caso, generaremos via “Exceptions” los errores que consideremos oportunos, aparte de errores de “RunTimeException” no controlados.

Ejemplo:

```
public void accionEnElService(Long id, ClaseDTO dto) {  
  
    Usuario usuario = repo.findById(idUsuario)  
        .orElseThrow(() -> new RuntimeException("Usuario no encontrado"));  
  
    if (comprobación_de_password) {  
        throw new PasswordIncorrectaException("La contraseña actual no es  
correcta");  
    }  
  
    ... Resto de código  
}
```

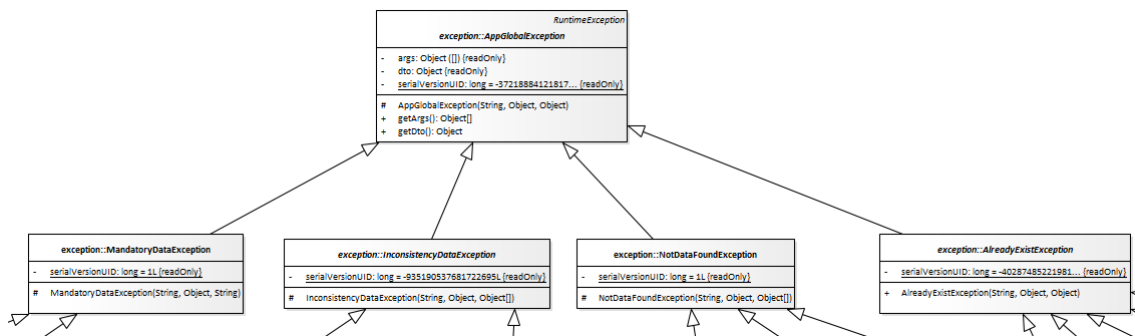
Para evitar “múltiples” try/catch en el Controller, y sobre todo, la famosa página “**ERROR 500**”, se ha implementado un manejador Global de excepciones en la clase `GlobalExceptionHandler`, anotado como `@ControllerAdvice`.

Compartidas por todas las clases anotadas como `@Controller`.



Se incorpora a esta clase el “MessageSource” para poder devolver el mensaje en el “Locale” del usuario.

Esta clase manejará las excepciones de nuestra clase principal de Excepciones “AppGloalException” (clase abstracta)



De la que hemos definidos 4 agrupaciones:

- **MandatoryDataException** Para los datos que son obligatorios.
- **InconsistencyDataException** Valores no coinciden con los esperados.
- **NotDataFoundException** Valores no encontrados.
- **AlreadyExistException** Valores ya existentes.

A partir de esta clase, se han “centralizado” todas las Excepciones propias en

```
try {
    modelService.grabaDatos(dto);
}
catch (UsernameNotFoundException ex) {
    result.rejectValue("errorGlobal", "error.en properties",
ex.getMessage());
}
catch (UserPasswordNotMatchException ex) {
    result.rejectValue("oldPassword", "error.enproperties",
ex.getMessage());
}
```

De esta forma al validar “`result.hasErrors`”, nos devolverá VERDADERO, con lo que deberemos volver al formulario y mostrar todos los mensajes generados.

- Mensajes de éxito con PRG (Post-Redirect-Get)
- Mensajes de error en la misma vista

FileStorageService

Descripción general del servicio

FileStorageService es un servicio encargado de gestionar el almacenamiento físico de archivos en el sistema de ficheros del servidor. Su responsabilidad principal es **guardar imágenes y documentos** asociados a entidades tales como Curso o de EstudianteCursosdominio, manteniendo una arquitectura desacoplada y fácilmente testeable.

Este servicio centraliza:

- Creación automática de directorios externos
- Generación de nombres únicos
- Validación básica del archivo
- Copia segura al sistema de ficheros
- Devolución de la ruta o nombre final almacenado

El servicio utiliza las rutas configuradas en AppProperties, evitando valores hardcodedos y permitiendo despliegues portables.

AppProperties

Clase utilitaria para encapsular los valores del fichero de configuración `application.properties`.

Nos permite inyectar en las clases que lo necesiten los valores definidos en la configuración que podrá y normalmente lo es, diferente entre los diferentes entornos (loca/desarrollo, contenedores (Docker, etc) y Producción si fuera necesario.

- ❓ **Cambiar rutas sin tocar código**
- ❓ **Configurar rutas distintas por entorno (dev, prod)**
- ❓ **Mantener el código limpio y portable**

Gestión Universal de URLs e i18n

1. El Problema Original

El cambio de idioma en aplicaciones con filtros (como nuestro buscador de cursos) fallaba porque:

- Los enlaces estáticos perdían los parámetros de búsqueda del usuario.
- Thymeleaf tiene restricciones de seguridad para acceder a la petición actual desde fragmentos (como el header).
- La concatenación manual de parámetros en el HTML es difícil de mantener y propensa a errores de sintaxis.

2. La Solución: ServletUriComponentsBuilder

Hemos utilizado una utilidad nativa de Spring Framework que vive en el ecosistema de servlets. Su función es actuar como un "clonador" de la petición HTTP actual.

A. Inyección Global (@ControllerAdvice)

Usamos un @ControllerAdvice para que esta herramienta esté disponible en todas las vistas sin tener que añadirla método a método en cada controlador.

Java

```
@ModelAttribute("urlBuilder")
```

```
public ServletUriComponentsBuilder urlBuilder() {  
    return ServletUriComponentsBuilder.fromCurrentRequest();  
}
```

- `fromCurrentRequest()`: Captura la URL completa, incluyendo el path (ej: /home) y todos los parámetros (ej: ?titulo=java&page=2).

B. Manipulación en la Vista (Thymeleaf)

En lugar de reconstruir la URL, le pedimos al objeto que modifique solo lo que necesitamos:

HTML

```
th:href="${urlBuilder.replaceQueryParam('lang', 'es').toUriString()}"
```

- `replaceQueryParam(key, value)`: Si el parámetro `lang` ya existe, lo cambia. Si no existe, lo añade. El resto de parámetros (filtros de búsqueda, nivel, etc.) se mantienen intactos.
- `toUriString()`: Convierte el objeto de nuevo en una cadena de texto válida para el atributo `href`.

CURSO

Subscripción

Evolución del curso

1. El cálculo del progreso (Backend)

Como el peso es relativo, tu fórmula en el Java (o en la Query) será algo como:

$$\text{Progreso} = \frac{\sum (\text{Peso de Tareas Completadas})}{\sum (\text{Peso Total de Tareas del Curso})} \times 100\%$$

2. El Botón Dinámico (Thymeleaf)

En la card del estudiante, sustituiremos los dos botones por uno solo que cambie según el estado del curso:

HTML

```
<div class="d-grid gap-2"> <div th:if="{curso.progreso == 100}" class="text-center">

    <button class="btn btn-outline-success w-100 fw-bold">

        <i class="bi bi-trophy-fill me-2"></i> Certificado / Finalizado

    </button>

</div>

<div th:if="{curso.progreso < 100}">

    <a th:href="@{/cursos/continuar/{id}(id={curso.id})}" class="btn btn-primary w-100">

        <i class="bi bi-play-circle me-2"></i> Continuar aprendiendo

    </a>

    <p th:if="{curso.tieneTareaPendiente}" class="text-muted small mt-1 mb-0 text-center">

        <i class="bi bi-clock-history"></i> Esperando nota del profesor...

    </p>

</div>
```

</div>

3. La Barra de Progreso con "Hitos"

Dado que usas módulos con peso, la barra de progreso "Slim" que comentamos antes se verá muy bien. Si quieres que sea más informativa, puedes añadir un pequeño efecto visual para indicar que hay algo "pendiente":

HTML

```
<div class="progress" style="height: 6px; border-radius: 0;">

  <div class="progress-bar bg-primary"

    th:style="'width: ' + ${curso.progreso} + '%" "></div>

    <div th:if="${curso.progresoPendiente > 0}"

      class="progress-bar bg-primary opacity-25"

      th:style="'width: ' + ${curso.progresoPendiente} + '%" "></div>

</div>
```

4. Resumen de UX para el Estudiante:

- Estado "Esperando Nota": Si el alumno ya terminó su parte pero el profesor no ha puntuado, es vital decírselo. Si no, verá que el progreso no sube y pensará que el sistema falla.
- El Trofeo: Cuando progreso == 100, el cambio de color de azul (acción) a verde o dorado (logro) da una descarga de dopamina necesaria para terminar el siguiente curso.

Un detalle sobre la "Última Tarea"

Si puedes pasar al modelo el nombre de la última tarea finalizada, podrías ponerlo debajo del título del curso en texto muy pequeño:

Último avance: Finalizaste "Módulo 2: Routing"

Esto ayuda al estudiante a ubicarse mentalmente nada más entrar al Dashboard.